

Clustering Exercises: Questions 5 and 10

Ivan Shyaka - 101229

September 2026

1 Question 5

This question asks what K-means with $K=2$ would find for the eight shoppers in Figure 12.16, depending on how the sock and computer purchase variables are scaled before clustering.

Raw number of items purchased. If clustering is done on the raw counts, socks purchased can range widely across shoppers while computers purchased only takes the value 0 or 1. That means the sock variable has far more spread, so it dominates the distance calculation K-means relies on. In this case, the clusters would essentially split shoppers into "bought few socks" versus "bought many socks," almost ignoring whether a computer was purchased at all, since that variable barely moves the distances between points.

Variables scaled to standard deviation one. Once both variables are scaled so each has a standard deviation of 1, they carry roughly equal weight in the distance calculation. Socks no longer dominate just because of a wider natural range. In this version, the clustering would end up more balanced between the two variables, and the split between shoppers who did or did not buy a computer starts to matter much more than it did with the raw counts.

Dollars spent. Computers cost far more than socks, so once purchases are converted to dollars, a single computer purchase adds a huge amount to that shopper's total compared to any number of sock purchases. The dollar-spent variable is then completely dominated by the computer purchase. The resulting clusters would basically separate shoppers into "bought a computer" versus "did not," regardless of how many socks they bought, since the sock spending is too small to shift the clustering either way.

In short, the same two shoppers can end up grouped completely differently depending on the scale used: raw counts group by sock quantity, standardized variables balance both purchases, and dollar amounts group almost entirely by the computer purchase. This is why the choice of scaling before K-means is not a minor detail, it can change what "similar" even means for the data.

2 Question 10

This exercise builds a simulated dataset with three known classes, then checks how well PCA and K-means can recover that structure without ever being told the true labels. All work was done in Python with numpy, scikit-learn, and pandas.

(a) Generating the simulated data

I generated 60 observations across 50 variables, split evenly into three groups of 20. Each group started from the same normal distribution, then I shifted the mean of each group so they would actually sit apart in space: class 1 stayed at 0, class 2 was shifted up by 3, and class 3 was shifted down by 3. Without that shift the three groups would just overlap as noise, so the shift is what gives them a real identity.

```
np.random.seed(1)
class1 = np.random.normal(0, 1, (20, 50))
class2 = np.random.normal(0, 1, (20, 50)) + 3
class3 = np.random.normal(0, 1, (20, 50)) - 3
X = np.vstack([class1, class2, class3])
true_labels = np.array([0]*20 + [1]*20 + [2]*20)
```

Resulting shape: 60 rows by 50 columns, with a matching array of 60 true labels.

(b) PCA and the first two components

Running PCA down to two components and plotting them by class showed three clearly separated groups, no overlap between them.

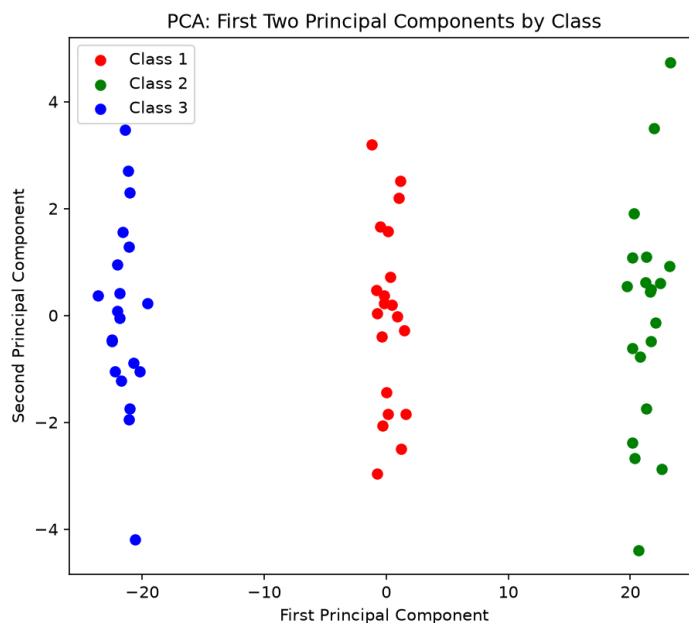


Figure 1: First two principal components, colored by true class

Since the classes were already well separated at this stage, there was no need to go back and increase the mean shift.

(c) K-means with $K = 3$

K-means on the raw 60 by 50 data recovered the three groups perfectly.

True Class	Cluster 0	Cluster 1	Cluster 2
0	0	0	20
1	20	0	0
2	0	20	0

Table 1: Crosstab of true class vs K-means cluster, $K=3$

Every observation from a given true class landed in a single cluster, with no mixing at all. The cluster numbers don't line up with the class numbers in order (class 0 mapped to cluster 2, for example), and that's expected, since K-means names its clusters arbitrarily. What matters is that each row has one clean, complete match, which means K-means rebuilt the exact same grouping as the true labels using no label information at all.

(d) K-means with $K = 2$

Forcing the algorithm down to only two clusters means two of the three real groups have to be merged.

True Class	Cluster 0	Cluster 1
0	20	0
1	20	0
2	0	20

Table 2: Crosstab of true class vs K-means cluster, $K=2$

True classes 0 and 1 were combined into a single cluster of 40 points, while class 2 stayed on its own. Looking back at the PCA plot, this makes sense: class 2 sat furthest away from the other two, so it kept its own identity, while the two closer classes got absorbed into one group once only two clusters were allowed. This shows what happens when you under-specify K : real structure gets lost because the model simply doesn't have enough groups to represent it.

(e) K-means with $K = 4$

Now the algorithm has one more cluster than it needs, and it responds by splitting one of the real classes in two rather than finding any new structure.

True Class	Cluster 0	Cluster 1	Cluster 2	Cluster 3
0	20	0	0	0
1	0	0	5	15
2	0	20	0	0

Table 3: Crosstab of true class vs K-means cluster, $K=4$

Classes 0 and 2 each stayed intact in their own cluster, but class 1 got split into two pieces (15 observations and 5). This is the opposite problem from $K=2$: instead of losing information by merging groups that should stay apart, $K=4$ manufactures an artificial split inside a group that was never meant to be divided, just to fill the extra cluster slot.

(f) K-means with $K = 3$ on the PCA scores

Rather than clustering the full 50-variable data, this time K-means only sees the 60 by 2 matrix of PCA scores from part (b).

True Class	Cluster 0	Cluster 1	Cluster 2
0	0	0	20
1	20	0	0
2	0	20	0

Table 4: Crosstab of true class vs K-means cluster on PCA scores

The result is identical in quality to part (c), a perfect match with zero mixing. This makes sense because the first two components already captured essentially all of the separation between classes, so reducing from 50 columns to 2 lost nothing that mattered for clustering, while cutting the computation down substantially.

(g) K-means with $K = 3$ on scaled data

Finally, scaling every variable to mean 0 and standard deviation 1 with StandardScaler before clustering gave the same outcome again.

True Class	Cluster 0	Cluster 1	Cluster 2
0	0	0	20
1	20	0	0
2	0	20	0

Table 5: Crosstab of true class vs K-means cluster on scaled data

Once again, a perfect match with no misclassification, the same as part (c). This is specific to how the data was built: every one of the 50 variables came from the same distribution and received the same size of shift per class, so no single variable had an outsized scale that could distort the distances K-means relies on. In a real dataset where variables are measured in different units, such as income versus age, scaling usually matters a great deal, since the variable with the largest raw numbers would otherwise dominate the clustering. Here the data was already balanced across variables, so scaling changed nothing.

Summary

Part	Setup	Result
(a)	Simulated 3 classes, 50 variables	60 by 50 dataset
(b)	PCA, first two components	Three clearly separated groups
(c)	K-means, K=3, raw data	Perfect match to true classes
(d)	K-means, K=2	Two true classes merged
(e)	K-means, K=4	One true class split in two
(f)	K-means, K=3 on PCA scores	Perfect match, same as (c)
(g)	K-means, K=3, scaled data	Perfect match, same as (c)

Table 6: Overview of all parts and their results

Appendix

Full code is submitted separately as a Jupyter notebook. Software used: Python 3, numpy, scikit-learn, pandas, matplotlib.